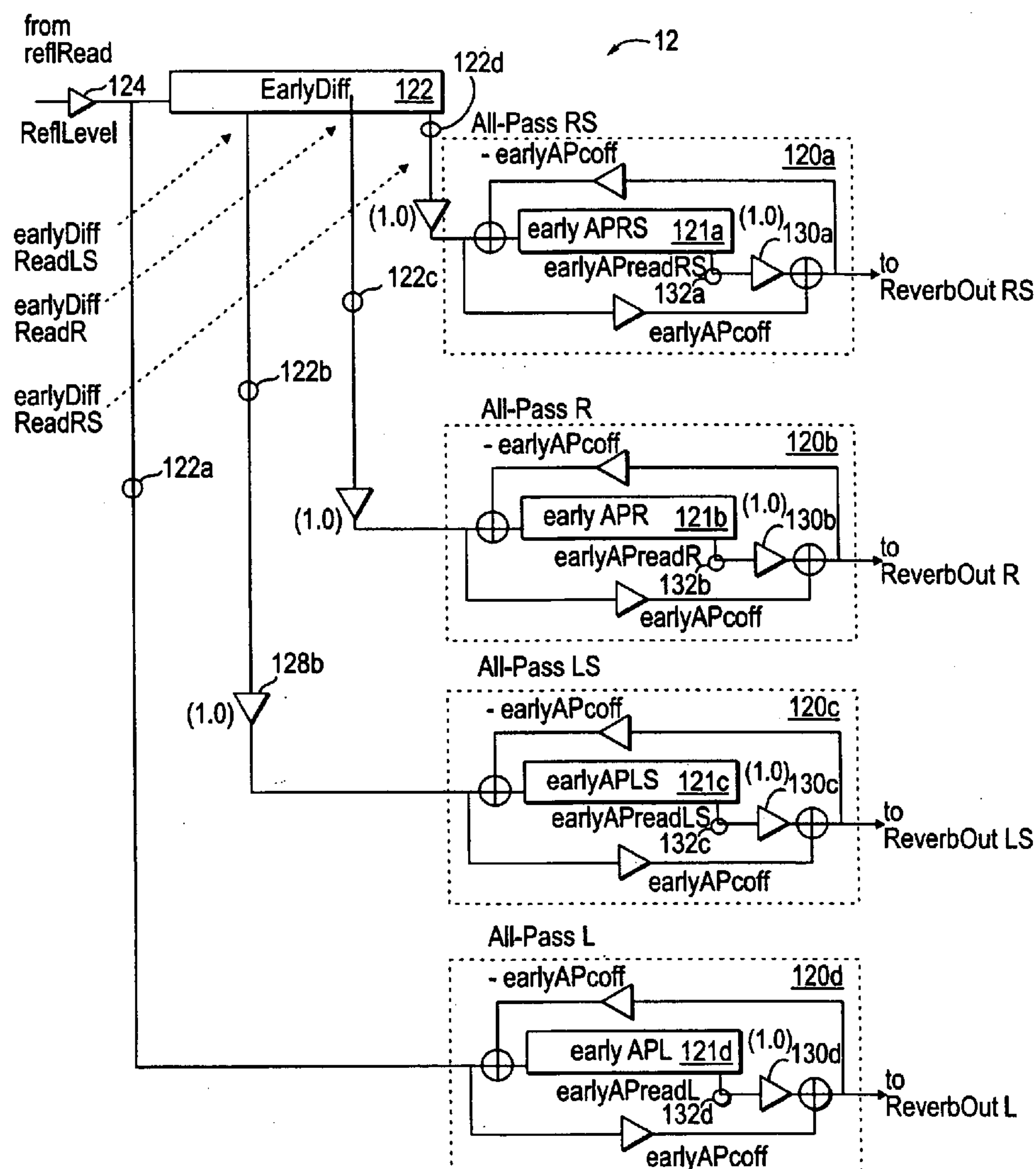


US 20040213416A1

(19) **United States**(12) **Patent Application Publication**
Dahl et al.(10) **Pub. No.: US 2004/0213416 A1**(43) **Pub. Date: Oct. 28, 2004**(54) **REVERBERATION PROCESSOR FOR
INTERACTIVE AUDIO APPLICATIONS****Publication Classification**(51) **Int. Cl.⁷** **H03G 3/00**(52) **U.S. Cl.** **381/63**(76) Inventors: **Luke Dahl**, Santa Cruz, CA (US);
Jean-Marc Jot, Aptos, CA (US);
Vincent Vu, Berkeley, CA (US); **Dana
Massie**, Santa Cruz, CA (US)(57) **ABSTRACT**

A reverberation processor is provided comprising a plurality of delay lines implemented in delay line memory, and hardware operable to introduce an echo effect into the reverberation decay and to provide a control parameter to control salience of the echo effect. The delay lines are operable to generate a reverberation decay in response to an input signal. In one embodiment, an increase and a decrease in the salience of the echo effect is dependent upon the control parameter. The control parameter may provide continuous control over the salience of the echo effect and the echo effect may be embedded within the reverberation decay. In one embodiment, a range of delay lengths across which the plurality of delay lines is distributed is reduced to produce a repeating echo effect. The invention extends to method of and software product for providing reverberation.

Correspondence Address:

BLAKELY SOKOLOFF TAYLOR & ZAFMAN
12400 WILSHIRE BOULEVARD
SEVENTH FLOOR
LOS ANGELES, CA 90025-1030 (US)(21) Appl. No.: **10/852,941**(22) Filed: **May 24, 2004****Related U.S. Application Data**(63) Continuation of application No. 09/547,365, filed on
Apr. 11, 2000.

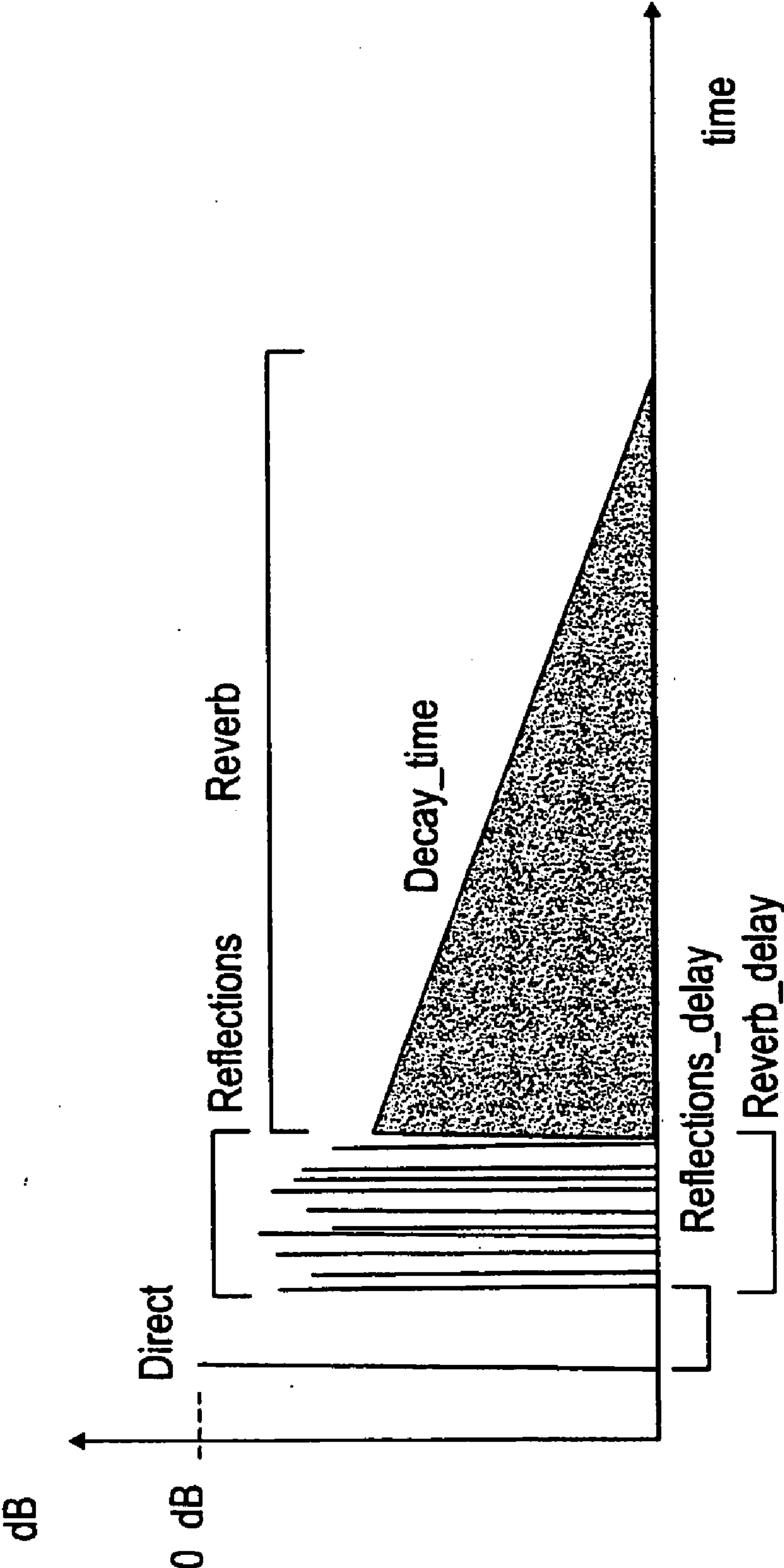


Fig. 1

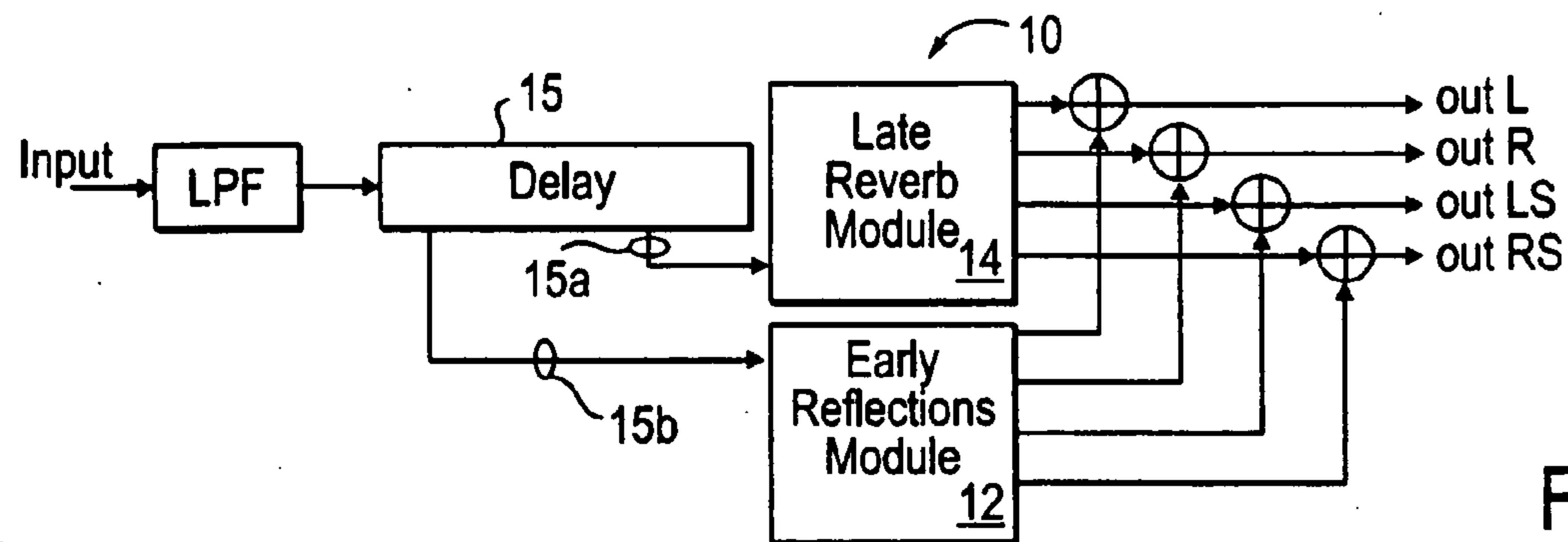


Fig. 2a

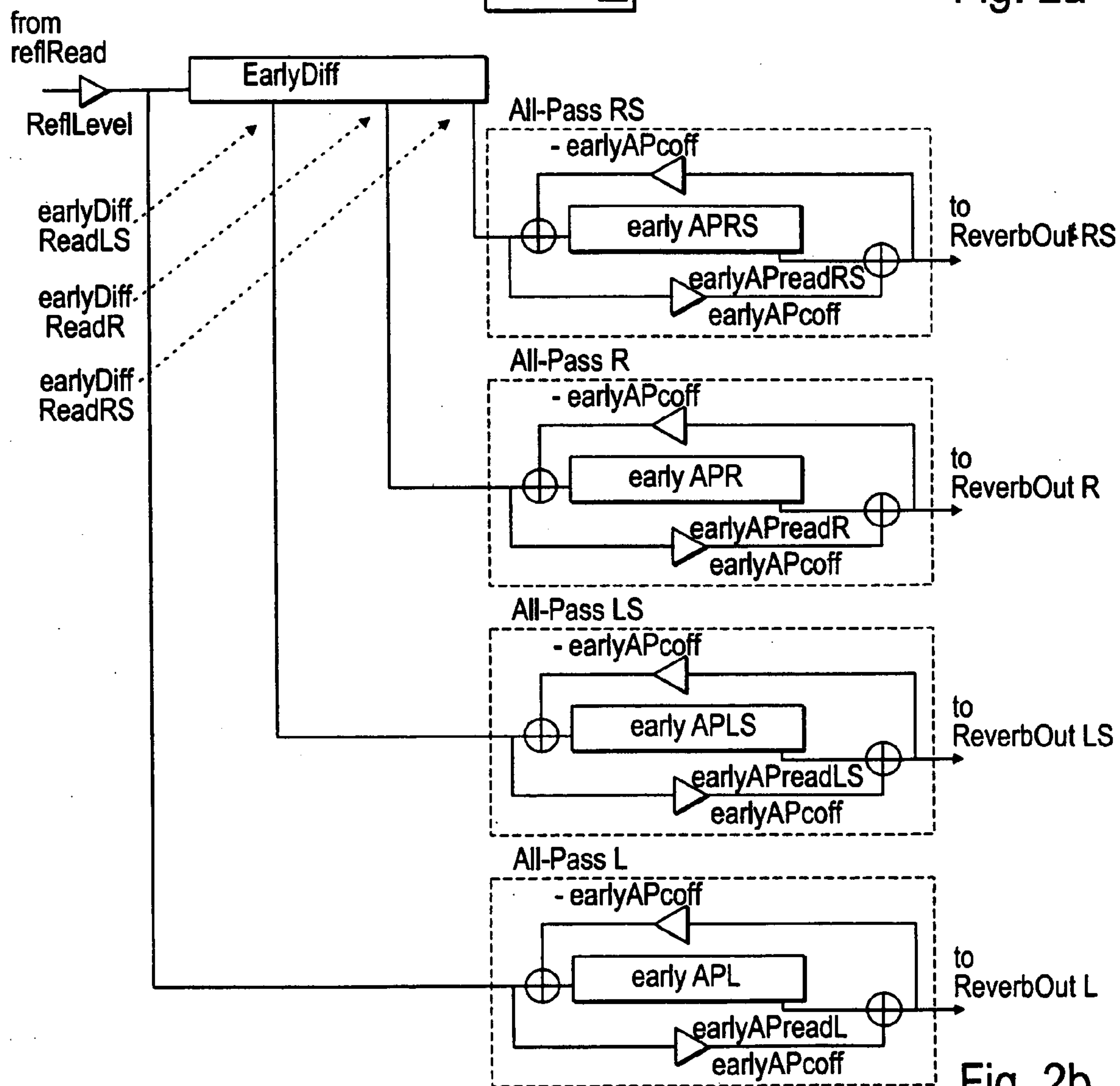


Fig. 2b

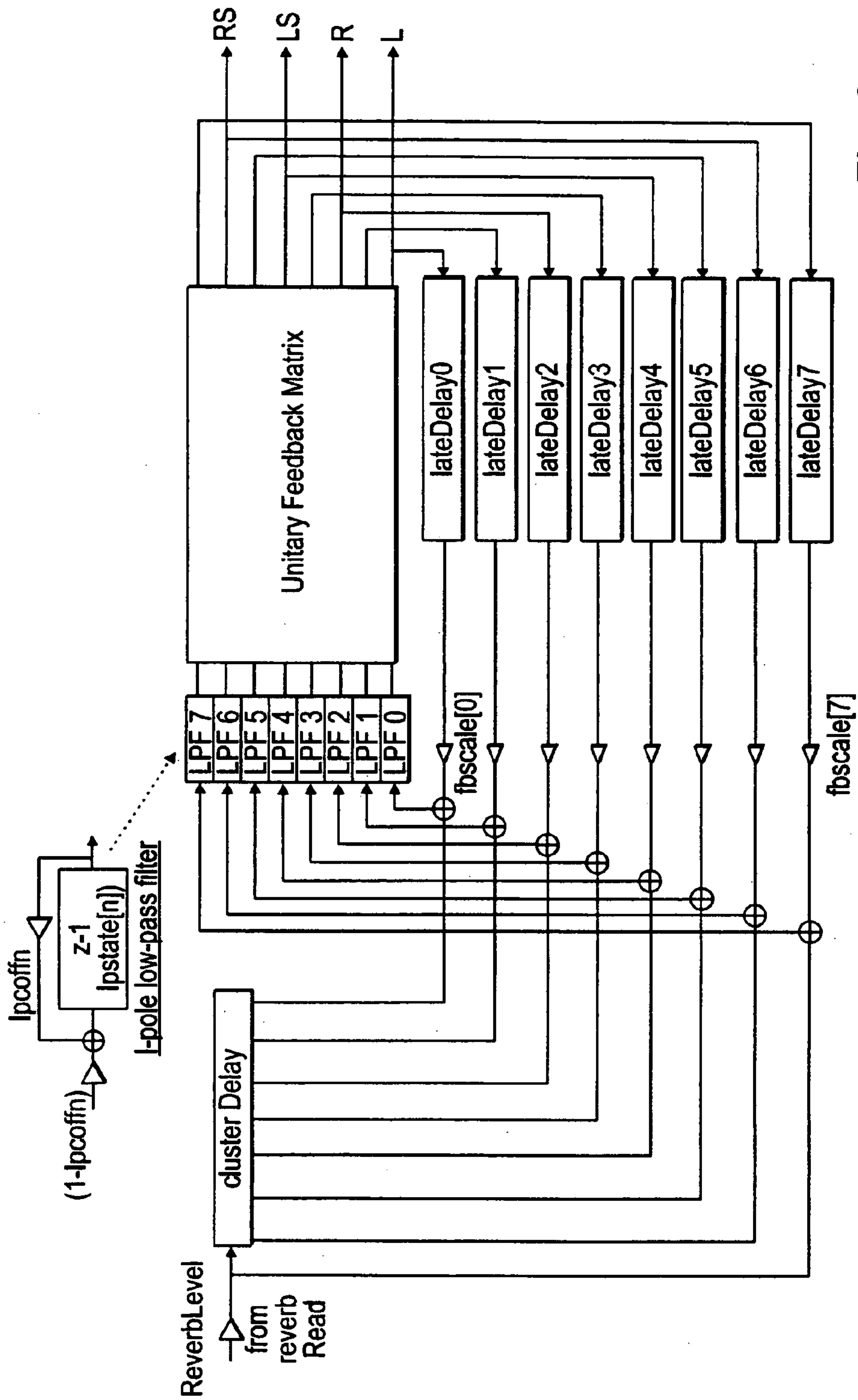


Fig. 2c

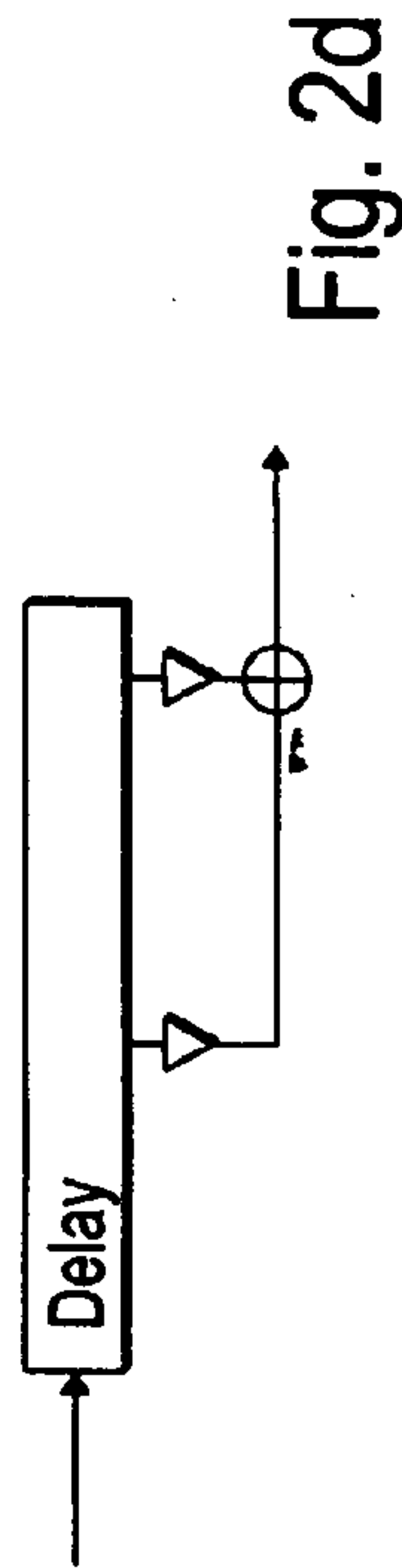


Fig. 2d

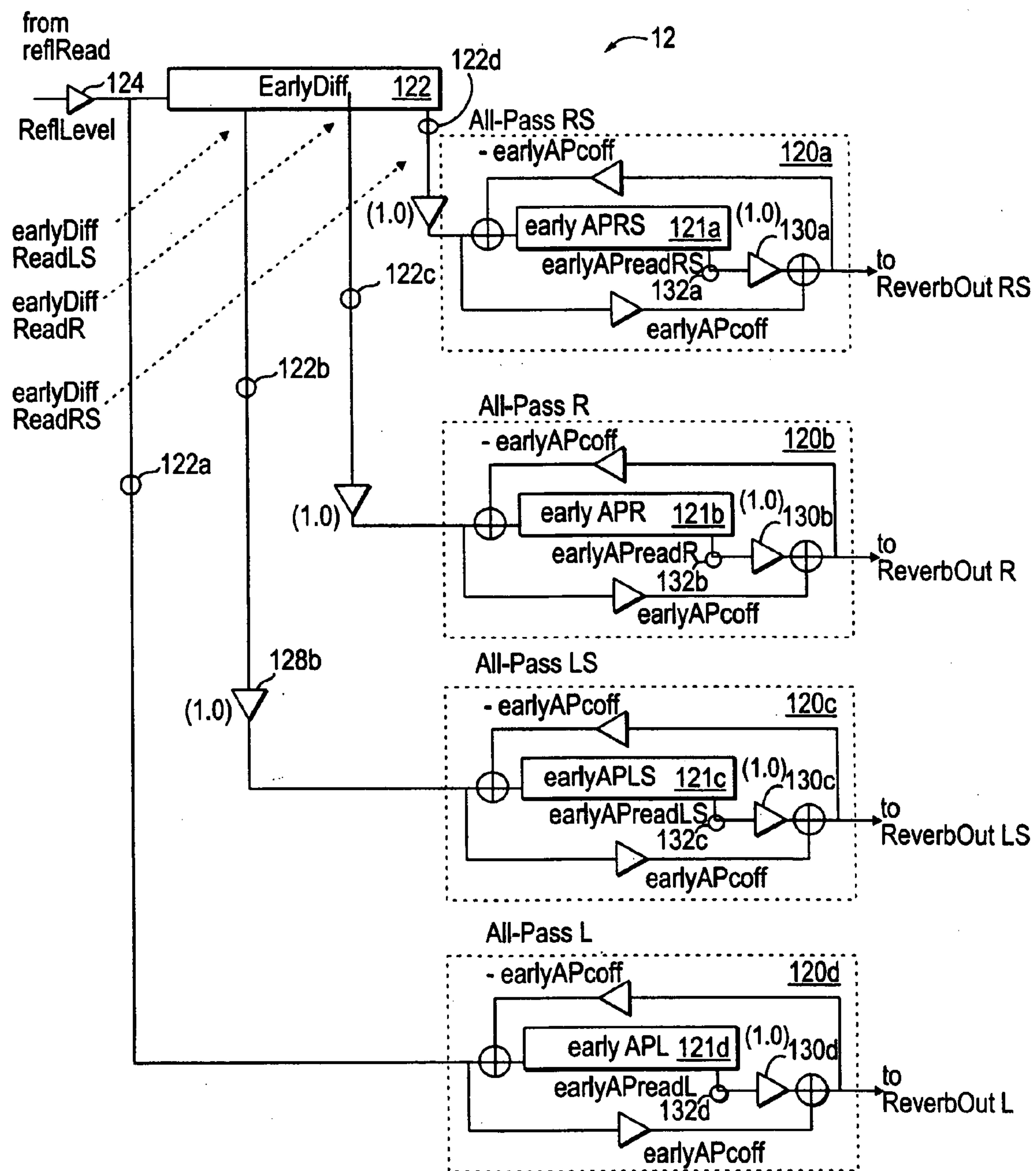


Fig. 3

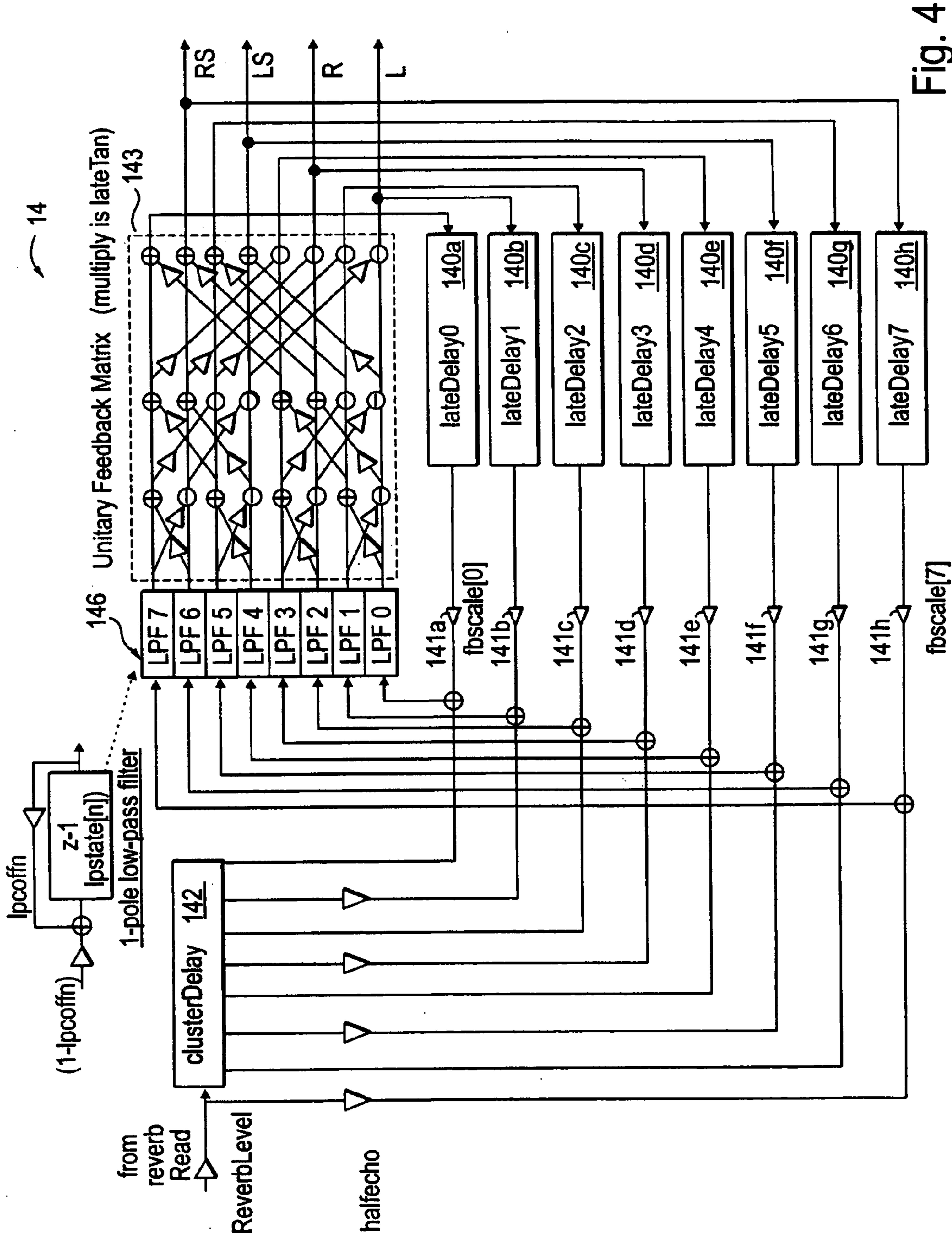


Fig. 4

REVERBERATION PROCESSOR FOR INTERACTIVE AUDIO APPLICATIONS

RELATED APPLICATIONS

[0001] This application is a continuation of U.S. application Ser. No. 09/547,365 filed Apr. 11, 2000.

BACKGROUND OF THE INVENTION

[0002] Virtual auditory displays (including computer games, virtual reality systems or computer music workstations) create virtual worlds in which a virtual listener can hear sounds generated from sound sources within these worlds. In addition to reproducing sound as generated by the source, the computer also processes the source signal to simulate the effects of the virtual environment on the sound emitted by the source. In a first-person computer game, the player hears the sound that he/she would hear if he/she were located in the position of the virtual listener in the virtual world. One important environmental factor is reverberation, which refers to the reflections of the generated sound which bounce off objects in the environment. Reverberation can be characterized by measurable criteria, such as the reverberation time, which is a measure of the time it takes for the reflections to become imperceptible. Computer generated sounds without reverberation sound dead or dry.

[0003] Artificial reverberation algorithms are well known in the art and are described e. g. in Stautner, J., and Puckette, M. (1982). Designing Multi-Channel Reverberators. *Computer Music Journal*, Vol. 6, no. 1, Dattorro, J. (1997). Effect Design (Part 1: Reverberator and Other Filters; Part 2: Delay-Line Modulation and Chorus). *Journal of the Audio Engineering Society*, Vol. 45, no. 9-10, Jot, J. -M. (1997). Efficient Models for Reverberation and Distance Rendering in Computer Music and Virtual Audio Reality. *Proceedings of the 1997 International Computer Music Conference*. The implementation of these algorithms on digital signal processors is based on a network of digital delay lines which are connected together and to the input and output points of the algorithm by feed-forward or feedback connections. Rooms of different sizes and acoustical properties can be simulated by modifying the topology of the network (the number of delay lines and the connections between them), by varying the duration of the delays, or by adjusting the amplification or attenuation coefficients of multipliers and filters inserted on the feed-forward or feedback connections.

[0004] As depicted in **FIG. 1**, a typical model of reverberation breaks the reverberation effects into discrete time segments. The first signal that reaches the listener is the direct-path signal, which undergoes no reflections. Subsequently, a series of discrete “early” reflections are received during an initial period of the reverberation response. Finally, after a critical time, the exponentially decaying “late” reverberation is modeled statistically because of the combination and overlapping of the various reflections. The magnitudes of Reflections_delay and Reverb_delay are typically dependent on the size of the room and on the position of the source and the listener in the room. As illustrated in **FIG. 2a**, the early reflections and the late reverberation are often generated by two separate processing modules whose output signals are combined to produce the output of the reverberation processor. Examples of an early reflection module and a late reverberation module are shown on **FIGS.**

2b and **2c**, respectively. The lengths of delay lines comprising these modules can be made smaller or larger according to the size of the virtual room.

[0005] Reverberation processors of the type described above are commonly used for the production of music and soundtracks in recording studios. In these applications, it is not common to produce drastic changes in reverberation characteristics while the sound is playing. Noticeable drop-offs and other artifacts in the output signal of the processor will occur, for instance, when the user loads a different reverberation “program” or adjusts the room size parameter (which may involve changing the network structure or modifying delay lengths). However, such artifacts are not acceptable in interactive audio applications. In immersive 3D games or simulation systems, for instance, different reverberation settings may be associated with different rooms or environments composing a virtual 3D world in which the virtual listener is allowed to travel. Consequently, in these systems, the reverberation processor must be able to change settings while creating a minimum of disruptive or distracting audible artifacts.

[0006] Artifacts due to dynamic changes in reverberation settings can be avoided by using two reverberation processors set to simulate different room acoustics and cross-fading from one processor to the other (at their input or at their output). However, it is generally more advantageous to use a single reverberation processor and modify its parameters in order to produce the desired change in room acoustics. The coefficients of multipliers and filters comprising a reverberation processor are easily changed, without noticeable artifacts, by ramping their values to new values over a short time. This avoids the introduction of sudden discontinuities in the audio signal waveform, audible as pops or clicks. For the same reason, it is necessary to avoid sudden changes in the duration of the delay lines.

[0007] Methods for implementing continuously variable delays are well known in the art and are described e. g. in Laakso, T. I. et al. (1996). Splitting the Unit Delay—Tools for Fractional Delay Filter Design. *IEEE Signal Processing Magazine*, Vol. 13, no. 1. However, these methods involve digital audio interpolators, adding significant computational complexity to each delay line. Furthermore, when an interpolator is used, a large variation in a delay length can produce a noticeable change in the pitch of the delayed signal, which may result in an audible artifact. Another technique for implementing variable delays is described in Van Duyne, S. A. (1997). A Lossless, Click-Free, Pitchbendable Delay Line Loop Interpolation Scheme. *Proceedings of the 1997 International Computer Music Conference* and illustrated in **FIG. 2d**. In this technique, a delay change is realized by cross-fading between two signals read from two different locations (or “taps”) in the delay line’s memory. These two taps are provided by two read pointers whose locations in the memory correspond to the original delay value and the final delay value. This method moves the read pointer to a new location in the delay memory without causing a drop-off, a discontinuity or a pitch alteration in the delayed audio signal. However, it causes a temporary timbre alteration known as “comb filter effect”.

[0008] In order to provide a wider range of variation in simulated room acoustics, it is common to allow control for the echo density of the reverberation decay, suggesting more

or less diffusing room walls. For instance, the reverberation algorithm described in Dattorro, *supra* allows to control diffusion by adjusting the feedback coefficients of a set of all-pass filters in the reverberation network. Another kind of common musical effect, also described e. g. in Dattorro *supra*, is the echo effect, which can be obtained simply by a single delay line with feedback. A cyclic echo can sometimes be obtained with existing reverberation algorithms as a side effect—usually unwanted—for particular settings of a reverberation processor's parameters. However, reverberators such as described in the above references do not provide parameters for controlling explicitly and intuitively aspects of an echo effect embedded in the reverberation decay. Such control would be useful to simulate larger rooms or semi-open environments such as a courtyard.

SUMMARY OF THE INVENTION

[0009] According to one aspect of the invention, audio artifacts are minimized when changing reverberation settings by causing the amplitude of a signal from a delay line having its read pointers changed to ramp down prior to moving the read pointer. The amplitude of the signal is then ramped up after the read pointer has been moved.

[0010] According to another aspect of the invention, a set of delay lines, whose output signals are combined to produce the output of a reverberation processor, are updated in sequence so that there is no audible drop-off in the processor's output signal.

[0011] According to another aspect of the invention, a reverberation processor provides continuous control over the salience of a periodically repeating echo in the late reverberation decay.

[0012] According to another aspect of the invention, a reverberation processor simultaneously provides continuous control over the salience of a periodically repeating echo in the late reverberation decay, and over the duration between successive repetitions of this echo.

[0013] According to another aspect of the invention, a reverberation processor simultaneously provides continuous control over the salience of a periodically repeating echo in the late reverberation decay, and over the "diffusion" (or echo density) of the reverberation decay. Furthermore, these two controls combine so that reducing the amount of diffusion has the effect of prolonging the audibility of the repeating echo along the reverberation decay.

[0014] Other features and advantages of the invention will be apparent in view of the following detailed description and appended drawings.

BRIEF DESCRIPTION OF THE DRAWINGS

[0015] FIG. 1 is a graph depicting the division of the reverberation response into early reflections and late reverberation;

[0016] FIG. 2a is a block diagram of a reverberation processor, made of an early reflections module and a late reverberation module;

[0017] FIG. 2b is a block diagram of an early reflections module;

[0018] FIG. 2c is a block diagram of a late reverberation module;

[0019] FIG. 2d is a block diagram of a variable delay line using a cross-fading technique;

[0020] FIG. 3 is a schematic diagram of a modified early reflections module according to the present invention; and

[0021] FIG. 4 is a schematic diagram of a modified late reverberation module according to the present invention.

DESCRIPTION OF THE SPECIFIC EMBODIMENTS

[0022] FIG. 2a depicts a standard reverberation processor. The reverberation processor 10 has one input and four outputs: Left, Right, Right Surround, and Left Surround. It has two primary components: the Early Reflections module 12, and the Late Reverb module 14. The input signal is low-pass filtered and then passes into a delay line 15 (roomDelay). The delay line 15 has two taps, reverbRead 15a and reflRead 15b, which feed the early reflections and reverberation module 12 and 14. Each of the four outputs of the Early Reflections module 12 is added to one of the four outputs of the Late Reverb module 14, and these signals are the reverberation outputs.

[0023] In a preferred embodiment of the present invention, the reverberation processor can be controlled by the following set of parameters (several of which would be affected by a simulated change of the room size):

[0024] Reflections Delay: the delay of the first early reflection relative to the direct-path signal.

[0025] Reverb Delay: the delay of the late reverberation onset relative to the first early reflection.

[0026] Reflections Level: the amplitude level of the early reflections.

[0027] Reverb Level: the amplitude level of the late reverberation.

[0028] Decay Time: the time it takes for the late reverberation to decay by 60 dB at low frequencies.

[0029] Decay HF Ratio: the ratio of the high-frequency decay time relative to the low-frequency decay time.

[0030] Modal Density: the total length of the delay lines comprising the Late Reverb module.

[0031] Diffusion: the echo density in the late reverberation.

[0032] Echo Depth: the salience of a repeating echo in the late reverberation.

[0033] Echo Time: the duration between successive repetitions of an echo in the late reverberation.

[0034] The problem of switching reverberation settings while creating minimal disruptive or disturbing artifacts is remedied in an embodiment of the present invention by the following mechanism. Any signals coming from delay line reads whose location might be changed are multiplied by variable coefficients. If the reverberation algorithm would not normally call for a multiply to be performed on that signal, it is multiplied by 1. When the delay line read pointer must be moved, the multiply coefficient is ramped towards zero. A short time later, when the coefficient has reached a sufficiently low value, the read pointer is moved, and the coefficient is then ramped back up to its correct value. If

multiple delay line read pointers must be moved, they are moved sequentially so that the audible impact of dips in the delayed signals is minimized at any moment.

[0035] This method of updating delay lengths in a reverberation processor is particularly efficient because:

[0036] the additional computational cost is limited, at most, to one multiplication per variable delay line read in the network, and,

[0037] a single ramper can be shared between all the delay line reads (since the read pointers are updated one at a time).

[0038] With this scheme, quick drops in audio parts of the reverberation are sometimes audible (the entire reverberation signal is never muted at once, but aspects of the sound may be heard to dip). Echoes are sometimes created when delay line read pointers are moved to much longer values. However, the overall effect is usually subtle and much less distracting than clicks and pops.

[0039] FIGS. 3 and 4 depict novel implementations of the early and late reverberation blocks 12 and 14 of the standard reverberation processor 10 depicted in FIGS. 2b and 2c. In a preferred embodiment, the reverberation block is implemented by software executed by digital signal processors (DSP). However, the implementation of the blocks will be depicted as schematic diagrams of hardware where the equivalence between the hardware and software is well known to persons of skill in the art.

[0040] For example, the delay lines depicted in FIGS. 2, 3 and 4 are implemented using samples stored in RAM. The roomDelay delay line 15 has two reads 15a and b which respectively feed the Early Reflections module 12 and Late Reverb module 14. The locations of the two read pointers are determined by the values of the Reflections Delay and Reverb Delay parameters supplied to the Reverb module. The read which feeds the Early Reflections, called reflRead, is set by Reflections Delay to be up to 300 msec after the start of the delay line. The read which feeds the Late Reverb module, called reverbRead, is set by Reverb Delay to be up to 100 msec after reflRead.

[0041] Early Reflections Module

[0042] FIG. 3 depicts a preferred embodiment of the Early Reflections module 12 which is made up of four parallel all-pass filters 120a-d which are fed by a tapped delay line 122 (EarlyDiff). Each all-pass filter 120 includes an all-pass delay line 121. The input signal, which comes from roomDelay delay line 15, is multiplied by the coefficient, ReflLevel 124. The value of this coefficient is set by the parameter Reflections Level to control the level of the early reflections in the reverberation sound. When Reflections Delay is modified, ReflLevel is ramped down, then the read pointer reflRead is moved to its new location in the delay line 15, and then ReflLevel is ramped back up.

[0043] After ReflLevel the signal enters the delay line EarlyDiff 122. EarlyDiff 122 has 4 taps 122a, 122b (earlyDiffReadLS), 122c (earlyDiffReadR), and 122d (earlyDiffReadRS) distributed across its length, which feed the four all-pass filters 120. The first tap, 122a, is at a fixed delay length of 0. The range of the other three tap delays changes proportionally with the amplitude of Reverb Delay.

[0044] The three signals which are read from the EarlyDiff delay line 122 are multiplied by first level setting coefficients

128b-c, set initially to 1, as indicated in FIG. 3. These first level setting coefficients 128 are used to ramp down the signal when the delay line read pointers in the EarlyDiff delay line 122 are moved.

[0045] The four all-pass filters 120a-d are identical except for the lengths of their all-pass delay lines 121a-d. The read pointers 132a (earlyAPreadRS), 132b (earlyAPreadR), 132c (earlyAPreadLS), and 132d (earlyAPreadL) to the delay lines 121 in the four all-passes 120 (and hence the effective length of the all-pass delays) are distributed and scaled proportionally to the amplitude of ReverbDelay. The signals which are read from the all-pass delay lines 121 are multiplied by second level setting coefficients 130a-d set initially to 1. These second level setting coefficients 130 are ramped down and then up when the all-pass delay lengths are changed, to avoid artifacts. The all-pass coefficient used in all four filters, named earlyAPcoeff, is set to the value of 0.4 (in this embodiment).

[0046] Late Reverb Module

[0047] FIG. 4 depicts a preferred embodiment of the Late Reverb module 14, made up of an 8-channel Feedback Delay Network (FDN) 140. The single input to the Late Reverb module, which comes from the reverbRead tap of the roomDelay delay line, feeds into the delay line clusterDelay 142. clusterDelay 142 has eight taps whose read pointer locations are constant and which feed the FDN.

[0048] There are eight delay lines 140a-h in the FDN 140, each having a different length. The total length of the delay lines 140a-h is specified by the Modal Density parameter. The outputs of all the lateDelay delay lines 140a-h are multiplied by the fbsscale coefficients 141, which are used to control the Decay Time of the reverberation and to normalize the Feedback Matrix 143. Once these eight signals have the delayed input signals added to them they are passed through 1-pole low-pass filters 146 and enter the feedback matrix 148. Each low-pass filter has its own filter coefficient (lpcoff0 for the 0th, etc.). The low-pass filter coefficients are adjusted according to the settings of the parameters Decay Time and Decay HF Ratio to control the decay time at high frequencies. Whenever one of the lateDelay read pointers 140a-h is updated, the corresponding fbsscale and lpcoff coefficients are updated according to the current settings of Decay Time and Decay HF Ratio.

[0049] Continuous Control of Diffusion in the Feedback Matrix

[0050] After the outputs of each lateDelay delay line have been multiplied by an fbsscale coefficient they are added to a delayed input signal, and filtered by a low-pass filter. The resulting signals are mixed together by a unitary mixing feedback matrix 143 before being fed back to the lateDelay delay line inputs. This feedback matrix can be changed from a diagonal matrix, which sends each of the inputs through unaffected, to a completely diffuse matrix, which mixes all of the input signals into each of the output signals. The amount of mixing affects the echo density of the reverb output, and is under control of the Diffusion parameter. A low Diffusion parameter value causes the feedback matrix to become diagonal, and a high value causes the matrix to become diffuse. An intermediate Diffusion value causes the matrix to be more or less diffuse.

[0051] The preferred implementation uses a recursive rotation matrix. A recursive rotation matrix can be made by

applying a 2×2 rotation matrix to each pair of inputs, and then applying the same rotation to the outputs of the first rotation until all of the inputs have been mixed into each of the outputs, as described in **FIG. 4**. The amount of mixing of the final 8×8 matrix can be controlled by one value, *lateTan*, which is calculated from the Diffusion parameter as follows:

[0052] The feedback matrix is made unitary by applying a normalizing gain to the values of the *fbscale* multipliers. This value is dependant on the diffusion of the feedback matrix and is calculated as follows:

$$FdnFbScale = \cos^3(\text{diffnorm} * \pi / 4)$$

[0053] Producing and Controlling a Repeating Echo in the Reverberation Decay

[0054] The lengths of the *lateDelay* delay lines **140** are distributed across a range of values. A repeating echo effect is achieved by reducing the range of lengths across which the delay lines are distributed. As the range becomes diminished, the repeating echo effect becomes more distinct.

[0055] The locations of the read pointers for the *lateDelay* lines in the FDN are determined by the values of Modal Density and Echo Depth according to the following equations (*DelayLen[i]* is the location in msec of the *i*th read pointer):

$$\text{In a preferred embodiment, } DMIN = 39.1 \text{ msec, } DMAX = 150.1 \text{ msec.}$$

[0056] As depicted in **FIG. 4**, *clusterDelay* **142** has eight taps whose locations are constant and which feed the FDN **140**. Four of these eight taps have signals are multiplied by the coefficient *halfecho* **145**. This coefficient is controlled by the parameter Diffusion, and is used in conjunction with *EchoDepth* to make echoes in the late reverb sound half as often. In a preferred embodiment:

$$\text{halfecho} = \text{diffnorm}$$

[0057] which has been calculated above.

[0058] The Sequence for Changing Reverberation Delays

[0059] There are five different parts of the reverberation processor which have delay line reads which must be updated without causing artifacts. In the preferred embodiment, many of the delay lines are modified when the Reverb Delay parameter is changed. This parameter controls the delay between the early reflections and late reverberation by changing the location of *reverbRead*. Changing the Reverb Delay parameter also spreads out the early reflections so that they span the time between Reflections Delay and the onset of the late reverberation. This is done by changing the *EarlyDiff* reads **122b-d**, and by changing the early all-pass delay lengths **121a-d**.

[0060] The parts that are changed are:

[0061] 1. The pointer *reflRead*, which reads from room-Delay **15** and feeds the Early Reflections module **12**, is moved when the Reflections Delay parameter is changed.

[0062] 2. The pointer *reverbRead*, which reads from room-Delay **15** and feeds the LateReverb module **14**, might be moved whenever Reflections Delay or Reverb Delay are changed.

[0063] 3. The three reads **122b-d** from the *EarlyDiff* delay **122** in the Early Reflections module are moved in sequence whenever Reverb Delay changes.

[0064] 4. The lengths of the four all-pass delays **121a-d** in the Early Reflections module **12** are changed in sequence whenever Reverb Delay changes.

[0065] 5. The *lateDelay* reads **140a-h** in the Late Reverb module **14** are moved in sequence whenever the Modal Density, the Echo Depth or the Echo Time is changed.

[0066] The program which controls the DSP includes a function called *GlitchlessSequence* which performs the steps necessary to update all the aforementioned delay read pointers. *GlitchlessSequence* uses a timer callback when it is necessary to wait *CBTIME* milliseconds for the next step. The delay line reads are changed according to the following pseudo code sequence:

[0067] 1. Change *reflRead* **15b**;

[0068] 2. Change *reverbRead* **15a**;

[0069] 3. If Reverb Delay has been changed or is in the process of being changed Continue;

[0070] Else if *lateDelay* read pointers need to be changed go to step #12;

[0071] Else if Reflections Delay has been changed go to step #1.

[0072] Else Break;

[0073] 4. Change *DiffReadR* **122c**;

[0074] 5. Change *DiffReadLS* **122b**;

[0075] 6. Change *DiffReadRS* **122d**;

[0076] 7. Change *earlyAPreadL* **132d**;

[0077] 8. Change *earlyAPreadR* **132b**;

[0078] 9. Change *earlyAPreadLS* **132c**;

[0079] 10. Change *earlyAPreadRS* **132a**;

[0080] 11. If *lateDelay* read pointers need to be changed Continue;

[0081] Else if Reflections Delay has been changed go to step #1;

[0082] Else if Reverb Delay has been changed go to step #2;

[0083] Else Break;

[0084] 12. Change *lateDelay* read pointer 0, **140a**;

[0085] 13. Change *lateDelay* read pointer 1, **140b**;

[0086] 14. Change *lateDelay* read pointer 2, **140c**;

[0087] 15. Change *lateDelay* read pointer 3, **140d**;

[0088] 16. Change *lateDelay* read pointer 4, **140e**;

[0089] 17. Change *lateDelay* read pointer 5, **140f**;

[0090] 18. Change *lateDelay* read pointer 6, **140g**;

[0091] 19. Change lateDelay read pointer 7, 140h;

[0092] 20. If Reflections Delay has been changed go to step #1;

[0093] Else if Reverb Delay has been changed go to step #2;

[0094] Else if lateDelay read pointers need to be changed go to step #12;

[0095] Else Break;

[0096] Each of these steps is composed of a couple smaller steps similar to those described in the section Specifics of Updating One Delay Line Read Pointer.

[0097] If a change is made to Reflections Delay and the sequence is not in progress GlitchlessSequence will be called starting at step #1. If the sequence is in progress a flag will be set which is used by steps #3, #11, and #20 to make sure the correct delay lines are changed the next time through the sequence. Similarly, changes to Reverb Delay and changes to the lateDelay read pointers (caused by changes in Modal Density, Echo Depth or Echo Time) will start the GlitchlessSequence at steps #2 and #12, respectively, or, if the sequence is already in progress, set their own flags.

[0098] The Specifics of Changing One Delay Line Read Pointer

[0099] The reverberation processor uses an interpolate instruction to ramp a coefficient for one value, called Ramp1Sub, to another value, called Ramp1Dest, at a rate determined by a variable called RampConst. For example, when the ReflRead tap to the roomDelay delay line 15 is to be moved then the ReflLevel coefficient would be initially equal to Ramp1Sub and would be ramped as described below.

[0100] Specifically, at each sample period:

$$\text{Ramp1Sub} = (\text{RampConst} * \text{Ramp1Sub}) + ((1 - \text{RampConst}) * \text{Ramp1Dest})$$

[0101] By replacing the ReflLevel coefficient with the new value Ramp1Sub at each sample period, the delayed signal can be ramped down or up, depending on the value of Ramp1Dest.

[0102] The value of RampConst determines how fast Ramp1Sub approaches the value of Ramp1Dest, and therefore how much time each step of the ramping sequence will take. For a given ramp time in milliseconds, say CBTIME, RampConst is set to:

$$\text{RampConst} = 2^{(-15 / (\text{CBTIME} * \text{SampleRate} / 1000))}$$

[0103] If Ramp1Dest were set to zero, for example, it would take Ramp1Sub CBTIME milliseconds to reach 15 bits below its original value. In the preferred embodiment CBTIME is 20 milliseconds.

[0104] The primary steps to move one delay line read pointer, reflRead for example, are as follows: ramp down ReflLevel, move reflRead, and ramp ReflLevel back up. However, to accomplish these steps there is a more detailed sequence of events that must take place.

[0105] 1. Set Ramp1Dest to the value of ReflLevel. This must be done so that the ramper does not ramp Ramp1Sub away before we can substitute it in for ReflLevel.

[0106] 2. Set Ramp1Sub to the value of ReflLevel, so that there will be no level jump when the substitution is made.

[0107] 3. Replace ReflLevel with Ramp1Sub in the instruction that multiplies the signal read from reflRead.

[0108] 4. Set Ramp1Dest to zero, so that the ramper starts ramping down.

[0109] 5. Wait CBTIME milliseconds while Ramp1Sub ramps down.

[0110] 6. Move reflRead to the new location.

[0111] 7. Set Ramp1Dest to the value of ReflLevel to start ramping up.

[0112] 8. Wait CBTIME milliseconds while Ramp1Sub ramps up.

[0113] 9. If ReflLevel has been changed since step 7,

[0114] Set ReflLevel to the value of Ramp1Sub,

[0115] Replace Ramp1Sub with ReflLevel in the multiply instruction,

[0116] Call the routine that would normal ramp ReflLevel if there were no update of the reflRead pointer.

[0117] 10. If ReflLevel has not been changed, replace Ramp1Sub with ReflLevel in the multiply instruction

[0118] If another delay line read pointer must be changed it can begin its own similar sequence immediately. It is not necessary to wait another time step.

[0119] The invention has now been described with reference to the preferred embodiments. Alternatives and substitutions will now be apparent to persons of skill in the art. For example, the particular ramping algorithm or delay configurations described can be modified by persons of skill in the art while practicing the principles of the invention. Accordingly, it is not intended to limit the invention except as provided by the appended claims.

What is claimed is:

1. A reverberation processor comprising:

a plurality of delay lines implemented in delay line memory, the delay lines being operable to generate a reverberation decay in response to an input signal; and

hardware operable to introduce an echo effect into the reverberation decay and to provide a control parameter to control salience of the echo effect.

2. The reverberation processor of claim 1, wherein an increase and a decrease in the salience of the echo effect is dependent upon the control parameter.

3. The reverberation processor of claim 1, wherein the control parameter provides continuous control over the salience of the echo effect.

4. The reverberation processor of claim 1, wherein the echo effect is embedded within the reverberation decay.

5. The reverberation processor of claim 1, wherein a range of delay lengths across which the plurality of delay lines is distributed is reduced to produce a repeating echo effect.

6. The reverberation processor of claim 5, wherein the repeating echo effect is made more distinct by further decreasing the range of lengths of the plurality of delay lines.

7. The reverberation processor of claim 1, wherein the control parameter is an echo depth control parameter and locations of read pointers of the plurality of delay lines are based on the echo depth control parameter.

8. The reverberation processor of claim 1, wherein the echo effect is a repeating echo pattern and an echo time control parameter controls a duration between successive repetitions of an echo.

9. The reverberation processor of claim 1, which comprises a feedback matrix connected to inputs and outputs of the plurality of delay lines, the feedback matrix being changed between a diagonal matrix and a diffuse matrix to change the echo effect in the reverberation decay.

10. The reverberation processor of claim 9, wherein the diffusion matrix changes the echo density in the reverberation decay.

11. The reverberation processor of claim 10, wherein the echo density is dependent upon a diffusion parameter, the changing of the feedback matrix being dependent upon the diffusion parameter.

12. The reverberation processor of claim 9, wherein the feedback matrix is a recursive rotation matrix.

13. The reverberation processor of claim 1, which is implemented using software.

14. A method of providing reverberation, the method comprising:

generating a reverberation decay in response to an input signal using a plurality of delay lines implemented in delay line memory; and

introducing an echo effect into the reverberation decay and controlling the salience of the echo effect using a control parameter.

15. The method of claim 14, wherein an increase and a decrease in the salience of the echo effect is dependent upon the control parameter.

16. The method of claim 14, wherein the control parameter provides continuous control over the salience of the echo effect.

17. The method of claim 14, wherein the echo effect is embedded within the reverberation decay.

18. The method of claim 14, wherein a range of delay lengths across which the plurality of delay lines is distributed is reduced to produce a repeating echo effect.

19. The method of claim 18, wherein the repeating echo effect is made more distinct by further decreasing the range of lengths of the plurality of delay lines.

20. The method of claim 14, wherein the control parameter is an echo depth control parameter and locations of read pointers of the plurality of delay lines are derived from the echo depth control parameter.

21. The method of claim 14, wherein the echo effect is a repeating echo pattern and an echo time control parameter controls a duration between successive repetitions of an echo.

22. The method of claim 14, which comprises:

applying delay line outputs of the plurality of delay lines to a feedback matrix;

applying feedback matrix outputs of the feedback matrix to delay line inputs of the plurality of delay lines; and

changing the feedback matrix between a diagonal matrix and a diffuse matrix to change the echo effect in the reverberation decay.

23. The method of claim 22, wherein the diffusion matrix changes the echo density in the reverberation decay.

24. The reverberation processor of claim 23, wherein the echo density is dependent upon a diffusion parameter, the changing of the feedback matrix being dependent upon the diffusion parameter.

25. The reverberation processor of claim 22, wherein the feedback matrix is a recursive rotation matrix.

26. A software product which, when executed by a digital signal processor, causes the processor to:

generate a reverberation decay in response to an input signal using a plurality of delay lines implemented in delay line memory; and

introduce an echo effect into the reverberation decay and provide a control parameter to control the salience of the echo effect.

27. The software product of claim 26, wherein an increase and a decrease in the salience of the echo effect is dependent upon the control parameter.

28. The software product of claim 26, wherein the echo effect is a repeating echo pattern and an echo time control parameter controls a duration between successive repetitions of an echo.

29. The software product of claim 26, wherein

delay line outputs of the plurality of delay lines are applied to a feedback matrix;

feedback matrix outputs of the feedback matrix are applied to delay line inputs of the plurality of delay lines; and

the feedback matrix is changed between a diagonal matrix and a diffuse matrix to change the echo effect in the reverberation decay.

30. A reverberation processor comprising:

means for generating a reverberation decay in response to an input signal using a plurality of delay lines implemented in delay line memory; and

means for introducing an echo effect into the reverberation decay and providing a control parameter to control the salience of the echo effect.

* * * * *